Data Structures and Algorithms

Lab 09 - Binary Search Trees (BST) and Advanced Tree Operations

Instructor: Sidrat-ul-Muntha

READ IT FIRST

Prior to start solving the problems in this assignments, please give full concentration on following points.

1. WORKING – This is individual lab. If you are stuck in a problem contact your teacher, but, in mean time start doing next question (don't waste time).

2. DEADLINE – One Week after assigned this.

3. SUBMISSION – This assignment needs to be submitted in a soft copy.

4. WHERE TO SUBMIT – Please visit your LMS.

5. **HOW TO SUBMIT** – All tasks must be completed in the **same Java file**, with Task 1's implementation reused in otter tasks. Clearly label Task 1, Task 2, and so on using comments in the code.

Learning Objectives

By the end of this lab, students will be able to:

- Implement a Binary Search Tree (BST) using a Node class.
- Perform insertion, searching, traversal, and deletion operations.
- Find minimum and maximum values in a BST.
- Validate whether a binary tree satisfies BST properties.
- Retrieve all BST values within a specified range.
- Determine whether a binary tree is a Complete Binary Tree.

Task 1: Implement a Binary Search Tree

Write a Java program to implement a Binary Search Tree using a Node class. Each node should contain an integer value and references to its left and right children.

Implement the following methods:

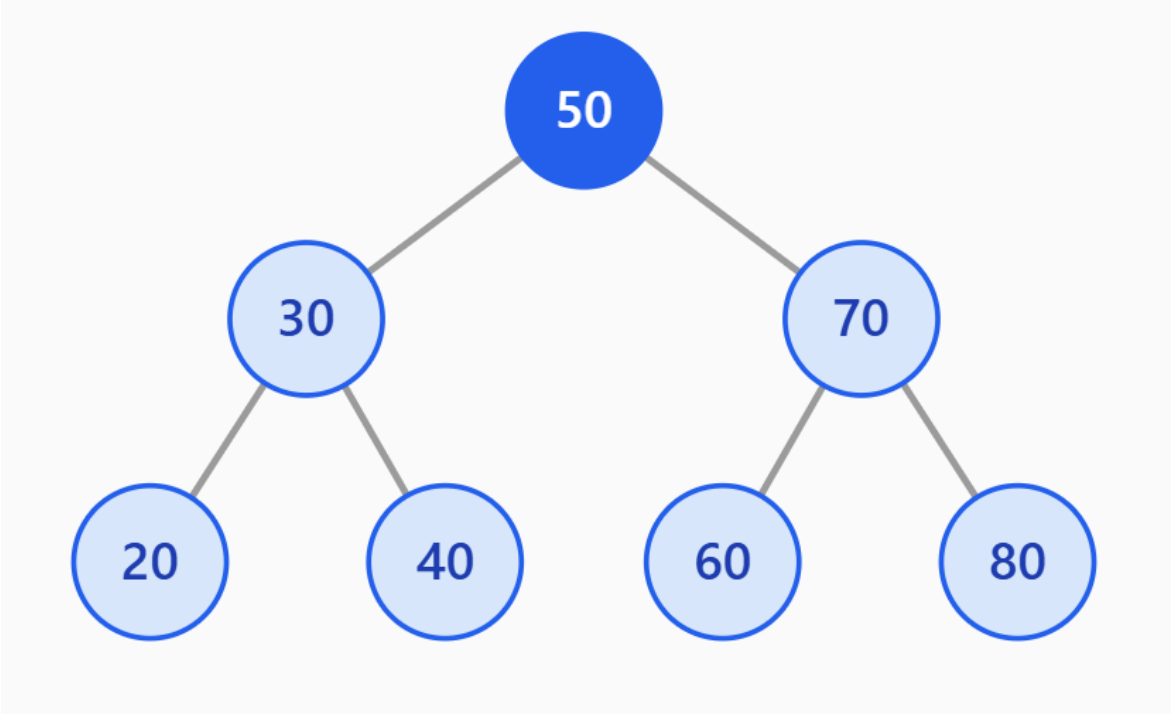
- **Node(int data)** – Create a new node with the given value.
- **insert(int value)** – Insert a new value into the correct position in the BST.
- **search(int value)** – Return true if the value exists; otherwise, return false.
- **delete(int value)** – Delete a specified node while maintaining BST properties.
- **inorder()** – Display nodes using inorder traversal.
- **preorder()** – Display nodes using preorder traversal.
- **postorder()** – Display nodes using postorder traversal.

The **delete()** method must handle three cases:

1. **Leaf Node:** Delete a node with no children.
2. **One Child:** Delete the node and connect its child to its parent.
3. **Two Children:** Replace the node with its inorder successor and delete that successor from its original position.

Demonstration
In the main() method, insert the following values:

50, 30, 70, 20, 40, 60, 80



Expected Output:

Inorder Traversal:

20 30 40 50 60 70 80

Preorder Traversal:

50 30 20 40 70 60 80

Postorder Traversal:

20 40 30 60 80 70 50

Searching for 40: Found

Searching for 60: Found

Searching for 90: Not Found

Initial BST:

20 30 40 50 60 70 80

Delete 20 (Leaf Node):

30 40 50 60 70 80

Delete 30 (One Child):

40 50 60 70 80

Delete 50 (Two Children):

40 60 70 80

Task 2: Find Minimum and Maximum Values in a BST

Extend the BST implementation from Task 1 to find the smallest and largest values.

Implement the following methods:

- findMin() – Return the minimum value in the BST.
- findMax() – Return the maximum value in the BST.

Your implementation must:

- Use BST properties to locate the minimum and maximum efficiently.
- Handle the case when the BST is empty.

Demonstration

Using the original BST before deletion:

Expected output:

Minimum Value: 20

Maximum Value: 80

Task 3: Validate a Binary Search Tree

Write a Java program to determine whether a given binary tree satisfies the properties of a Binary Search Tree.

Implement the following method:

- isValidBST() – Return true if the tree is a valid BST; otherwise, return false.

Demonstration

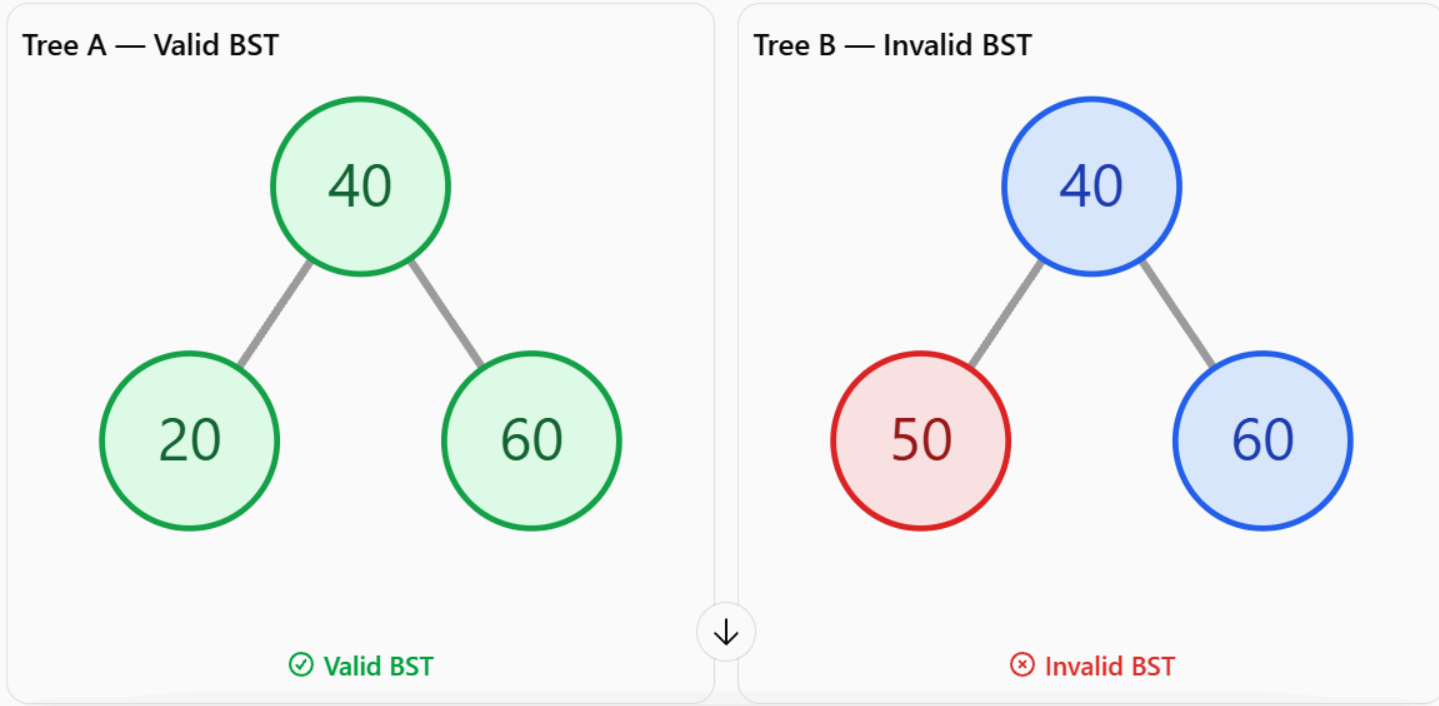
Your implementation must:

- Verify that all values in the left subtree are smaller than the parent node.
- Verify that all values in the right subtree are greater than the parent node.
- Check the entire subtree, not only immediate children.
- Reject duplicate values.
- Test both valid and invalid binary trees.

Expected output:

Tree A: Valid BST

Tree B: Invalid BST



Task 4: Find All Values Within a Given Range

Write a Java program to display all values within a specified range in a Binary Search Tree.

Implement the following method:

- printRange(int low, int high) – Display all values within the specified range in ascending order.

Your implementation must:

- Include values equal to the lower and upper boundaries.
- Use BST properties to avoid unnecessary subtree traversal.
- Display the results in sorted order.
- Handle cases where no values exist within the specified range.

Demonstration

Use the original BST containing:

50, 30, 70, 20, 40, 60, 80

Expected output:

Range [30, 70]:

30 40 50 60 70

Range [45, 75]:

50 60 70

Range [10, 25]:

20

Range [90, 100]:

No values found

Task 5: Check Whether a Binary Tree Is a Complete Binary Tree

Write a Java program to determine whether a given binary tree is a Complete Binary Tree (CBT).

A Complete Binary Tree is a binary tree in which all levels are completely filled except possibly the last level, and all nodes at the last level are positioned as far left as possible.

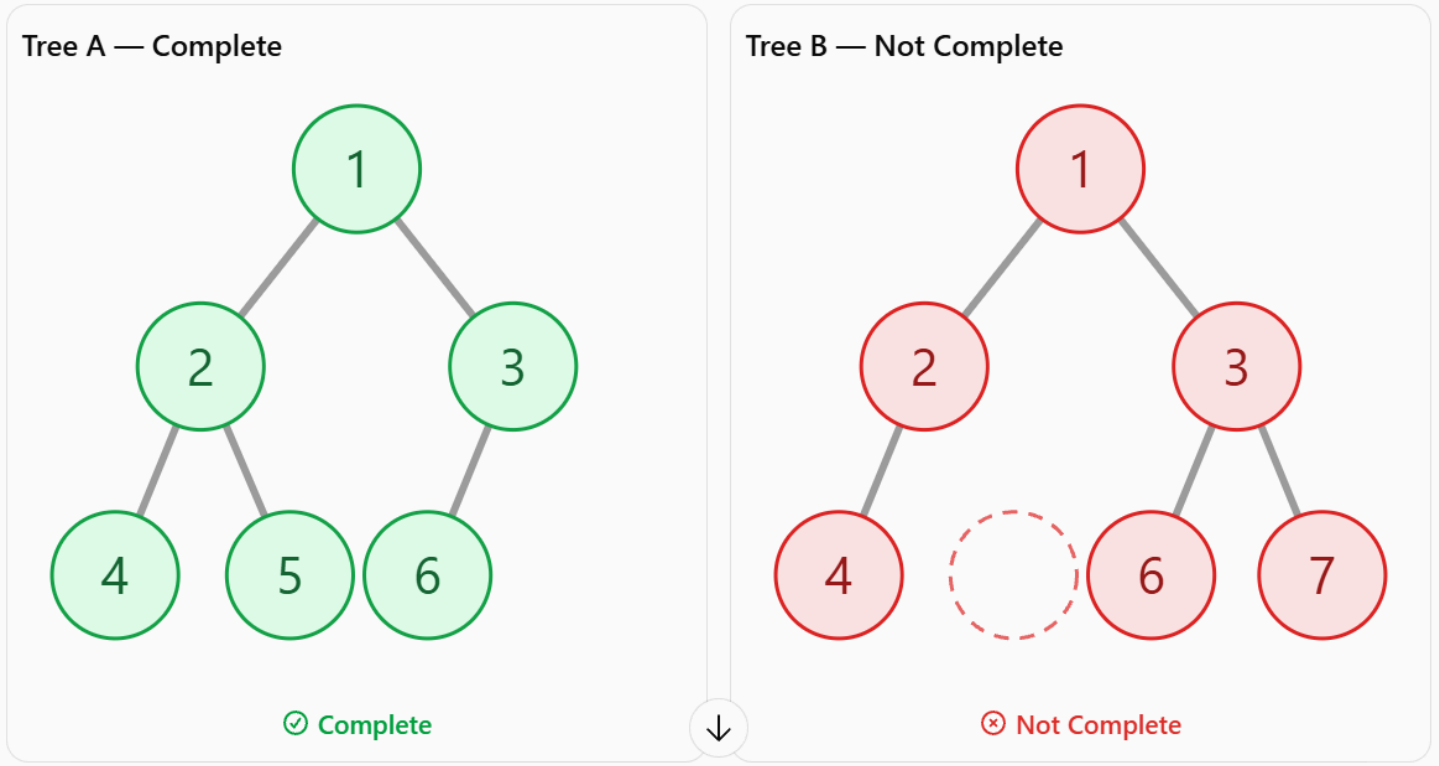
Implement the following method:

- isCompleteBinaryTree() – Return true if the binary tree is complete; otherwise, return false.

Your implementation must:

- Use level-order traversal (Breadth-First Search) with a queue.
- Check that nodes are filled from left to right.
- Identify missing child positions.
- Return false if a node appears after a missing child position during level-order traversal.
- Handle an empty tree correctly.

Demonstration



Expected output:

Tree A: Complete Binary Tree

Tree B: Not a Complete Binary Tree